

SIMATS
ENGINEERING



SIMATS
Saveetha Institute of Medical And Technical Sciences
(Declared as Deemed to be University under Section 3 of UGC Act 1956)

# PY THON PROGRAMMING

# ASSIGNMENT – 2


**Name** : **Hemnath R**

**RegNo** : **1920260116**


**Course Code** : **CSA0822**

**GitHUB Link** : https://github.com/hemnathr08-commits/python-programming-assignment_

# QUESTION 1: Decision Making – Electricity Subsidy Eligibility (6 Marks)

## 1.1 Required task

Develop a Pythonp rogram to decide electricity subsidy eligibility using conditional statements. The task compares separate **if** statements, an **if–elif–else** chain, nested conditions, Boolean expressions, **match-case**, and a lookup-list approach. Boundary conditions must be checked carefully.

## 1.2 Rules used

| Condition | Decision |
|---|---|
| Consumption up to and including 100 units | Full subsidy |
| Consumption from 101 to 200 units, with ration card | Partial subsidy |
| Consumption from 101 to 200 units, without ration card | No subsidy |
| Consumption above 200 units and age strictly greater than 8S0pecial exception | |
| All other cases | No subsidy |

**Boundary note:** “Strictly greater than 80” means age 80 does not qualify; age 81 does. “Above 200” means 200 itself is not above 200.

## 1.3 Recommended implementation

```
defsubsidy_decision(units, age,has_ration_card):
    if units < 0 or age < 0:
        return "Invalid input"

    ifunits <= 100:
        return "Full subsidy"
    elif 101 <= units <= 200 and has_ration_card:
        return "Partial subsidy"
    elif   units > 200 and age > 80:
          return "Special exception"
    else:
        return "No subsidy"


units = int(input("Enter electricity units: "))
age = int(input("Enter age: "))
answer = input("Do you have a ration card? (yes/no): ").strip().lower()
has_card = answer == "yes"

print("Eligibility:", subsidy_decision(units, age, has_card))
```

## 1.4 How the conditions work

The firstcondition handlesconsumptionof 100 units or less. The next condition checks both the consumption range and ration-card status. The special exception is checked after the ordinary low-consumption and ration-card rules. The final else handles every remaining valid case. An if–elif chain ensures that only one decision is returned.

## 1.5 Boolean expression

```
101 <= units <= 200 and has_ration_card
units > 200 and age > 80
```

The operator **and** requires both conditions to be true. A chained comparison such as **101 <= units <= 200** checks the lower and upper limits together.

## 1.6 Sample test cases

| Units | Age | Ration card | Expected decision |
|---|---|---|---|
| 80 | 35 | No | Full subsidy |
| 100 | 45 | Yes | Full subsidy |

| Units | Age | Ration card | Expected decision |
|---|---|---|---|
| 101 | 40 | Yes | Partial subsidy |
| 200 | 50 | No | No subsidy |
| 201 | 80 | No | No subsidy |
| 201 | 81 | No | Special exception |
| 250 | 90 | Yes | Special exception |
| -5 | 30 | No | Invalid input |

### 1.7 Alternative forms and complexity

Separate if statementsare independent, so more than one message may be printed if conditions overlap. An if–elif–else chain is preferable when exactly one result is required. Nested if statements can express the same rules but may be harder to read. A match-case statement is useful when matching a single category value; it is less natural for several numeric ranges. A lookup list can store range labels and decisions, but the conditions still need careful boundary handling.

For a fixed number of conditions, decision time is **O(1)** and auxiliary space is **O(1)**.

## QUESTION 2: Loops and Attempts – OTP Verification (4 Marks)

### 2.1 Required task

Create anOTP verification program that limits the number of attempts. Demonstrate the use of while and for loops and explain break, continue, pass, and the loop else clause. A security-question answer may provide a limited extra opportunity.

### 2.2 Program

```
 correct_otp       = “4826”
correct_security_answer  =  "blue"
max_attempts     =     3
verified = False

for attempt in range(1, max_attempts + 1):
    entered = input(f"Enter OTP (attempt {attempt}/{max_attempts}): ").strip()

    if not entered:
        print("Empty input. Try again.")
        continue

    if entered == correct_otp:
        verified = True
        print("OTP verified successfully.")
        break
    else:
        print("Incorrect OTP.")

else:
    print("OTP attempts exhausted.")
    security = input("Answer the security question: ").strip().lower()

    if security == correct_security_answer:
        extra = input("Enter the OTP once more: ").strip()
        if extra==  correct_otp:
            verified= True
            print("Verified using the extra attempt.")
        else:
            print("Verification failed.")
    else:
        print("Security answer incorrect. Verification failed.")

if not verified:
    print("Access not granted.")
```

### 2.3 Loop concepts

| Statement / construct | Purpose in this task |
|---|---|
| while | Repeats while a condition remains true; suitable when the number of attempts is condition-controlled. |
| for | Repeats over a known range of attempts. |
| break | Stops the loop immediately after successful verification. |
| continue | Skips the remaining statements for the current iteration, such as when input is empty. |
| pass | A placeholder statement that performs no operation; useful while developing an unfinished branch. |
| loop else | Runs only when the loop finishes normally without break. |

### 2.4 While-loop alternative

```
attempts = 0
verified = False

while attempts < 3 and not verified:
    attempts += 1
    entered = input("Enter OTP: ").strip()

    ifentered == "4826":
        verified = True
        print("OTP verified.")
    else:
        print("Incorrect OTP.")

if not verified:
    print("Maximum attempts reached.")
```

The example uses a fixed demonstration OTP and security answer. A real application must generate OTPs securely, expire them, avoid storing them as plain text, and enforce attempt limits on the server.

### 2.5 Complexity

Ifthemaximum number of OTP attempts is k, the loop performs at most **k** comparisons. Time complexity is **O(k)**; auxiliary space is **O(1)**.

# QUESTION 3: String Operations – Vehicle Registration Validator(6 Marks)

### 3.1 Required task

Normalisea registrationstring by removing permitted spaces and hyphens and converting letters to uppercase. Validatethe structureusingstring operations and slicing. The main implementation uses no regular expressions.

### 3.2 Assumed structure

Theaccepted structure istwoletters, two digits, one or two letters, and four digits: for example, **TN09AB1234** or **TN09A1234**. This is a structural format check only; it does not confirm that a registration number is officially issued.

### 3.3 Program

```
def validate_registration(raw):
    # Remove only the permitted separators and normalise case.
    value = raw.replace(" ", "").replace("-", "").upper()

    #Valid total lengths are 9 or 10 characters.
    iflen(value) not in (9, 10):
        return False, value

    #First two characters must be letters.
    ifnot value[0:2].isalpha():
    return False, value

    #Next two characters must be digits.
    ifnot value[2:4].isdigit():
    return False, value

    #The series must contain one or two letters.
```

```
    series_length = len(value) - 8
    ifseries_length not in (1, 2):
        return False, value

    series = value[4:4 + series_length]
    number = value[4 + series_length:]

     ifnot series.isalpha():
         return False, value

    iflen(number) != 4 or not number.isdigit():
        return False, value

    return True, value


registration = input("Enter vehicle registration: ")
valid, normalised = validate_registration(registration)

if valid:
    print("Valid structure:", normalised)
else:
    print("Invalid registration structure:", normalised)
```

### 3.4 String operations demonstrated

**replace()**removes spacesand hyphens;**upper()**normalises alphabetic characters; **len()** checks the total length; slicing extracts the region code, district digits, series and final number; **isalpha()** and **isdigit()** check character groups. Python strings are immutable. Methods such as upper() and replace() return new strings; they do not modify the original string in place.

### 3.5 Test cases

| Input | Normalised value | Expected result |
|---|---|---|
| tn 09-ab 1234 | TN09AB1234 | Valid structure |
| TN-09-A-1234 | TN09A1234 | Valid structure |
| T109AB1234 | T109AB1234 | Invalid: first two are not letters Invalid: |
| TNXXAB1234 | TNXXAB1234 | district is not numeric Invalid: final |
| TN09A12B4 | TN09A12B4 | section is not four digits Invalid: |
| TN09AB12345 | TN09AB12345 | incorrect length |

### 3.6 Alternative approach and complexity

Aregularexpressioncould express the same patterncompactly, but the main solution deliberately uses built-in string operations and slicing. For a string of length n, normalisation and validation each scan a bounded number of characters, so time complexity is **O(n)**. The normalised string requires **O(n)** additional space.

# QUESTION 4: String Building – +=, join() and f-strings (6 Marks)

### 4.1 Required task

Comparer epeated string concatenation using **+=** with collecting pieces and using **join()**. Demonstrate formatted outputwith f-strings and measureexecution time for 1,000, 10,000 and 200,000 pieces.

### 4.2 Benchmark program

```
import time

def build_with_plus(parts):
    result = ""
    for part in parts:
    result += part
    return result


def build_with_join(parts):
    return "".join(parts)
```

```
def measure(function, parts, repeats=3):
    # Return the best of repeated measurements to reduce random noise.
    best = float("inf")
    for _ in range(repeats):
        start = time.perf_counter()
        result = function(parts)
        elapsed = time.perf_counter() - start
        best = min(best, elapsed)
    return best, len(result)

sizes = [1_000, 10_000, 200_000]

for size in sizes:
    parts = ["Python"] * size
    plus_time, plus_length = measure(build_with_plus, parts)
    join_time, join_length = measure(build_with_join, parts)


    print(f"Pieces: {size:,}")
    {pprliunst_(tfi"m+e= :.t6ifm}e :seconds")
    print(f"join time: {join_time:.6f} seconds")
    print(f"Same output length: {plus_length == join_length}")
    print()
```

### 4.3 Formatted strings

```
student = "Arun"
mark = 92
print(f"Student: {student} | Mark: {mark}")
```

An f-string places expressions inside braces and is convenient for readable formatted output. It is appropriate for producing labels, reports and aligned summaries.

### 4.4 Comparison

| Method | Description | Performance consideration |
|---|---|---|
| result += part | Creates the growing result inside a loop. | May repeatedly copy earlier content; can approach $O(n^2)$ total work. |
| ''.join(parts) | Combines an iterable of strings in one operation. | Usually more efficient for many pieces; O(total output length). |
| f-string | Formats values into a string. | Best for readable formatting; not a replacement for joining a large collection. |

Actual timings depend on the Python version, operating system, processor and background activity. The program prints measured values; no fixed timing result should be assumed.

### 4.5 Complexity

Let mbethenumber of pieces and L the total number of output characters. join() takes approximately **O(m + L)** time and **O(L)** space for the result. Repeated += can require up to **O(mL)** copying in the general case, often described as quadratic when piece sizes are similar. CPython may optimise some concatenation patterns, so benchmarking is important.

# QUESTION 5: Functions and Scope – Canteen Order System (7 Marks)

### 5.1 Required task

Createafunction-based canteen order program and demonstrate positional, keyword and default parameters. Explain local,globalandnonlocal scope. Identifythemutable default argument problem and correct it by using None.

### 5.2 Mutable default argument problem

```
# Problematic: the same list is reused between calls.
def add_item_bad(item, order=[]):
    order.append(item)
    return order

print(add_item_bad("Tea"))     # ['Tea']
print(add_item_bad("Coffee"))  #['Tea', 'Coffee'] -- shared list!
```

Default argument values are created when the function is defined, not each time it is called. Therefore, a mutable list used as a default can retain data between calls.

## 5.3 Corrected canteen program

```
MENU = {
    "tea": 12.0,
    "coffee": 18.0,
    "sandwich": 45.0,
    "juice": 30.0
}

def add_item(item, quantity=1, order=None):
    if order is None:
        order = []

    item = item.strip().lower()
    ifitem  not in MENU:
        raise ValueError("Item is not available in the menu.")
    ifquantity  <= 0:
        raise ValueError("Quantity must be greater than zero.")

    order.append((item, quantity))
    return order

def calculate_total(order, discount=0.0):
    subtotal = sum(MENU[item] * quantity for item, quantity in order)
    if not 0 <= discount <= 100:
        raise ValueError("Discount must be between 0 and 100.")
    return subtotal * (1 - discount / 100)

my_order = []
add_item("tea", order=my_order)
add_item("sandwich", quantity=2, order=my_order)

total = calculate_total(my_order, discount=5)
print("Order:", my_order)
print(f"Total after discount: Rs. {total:.2f}")
```

## 5.4 Parameter types

| Type | Example | Meaning |
|---|---|---|
| Positional | add_item('tea', 2, my_order) | Arguments are matched by position. |
| Keyword | add_item(item='tea', quantity=2) | Arguments are identified by parameter name. |
| Default | add_item('tea') | quantity uses its default value of 1. |

## 5.5 Scope demonstration

```
message = "Global value"

def outer():
    count = 0              #local to outer()

    def inner():
        nonlocal count     #refers to outer()'s variable
        count += 1
        return count

    print(inner())
    print(inner())

outer()
print(message)
```

A local variable is created inside a function and is normally available only there. A global variable is defined at module level. The nonlocal keyword lets a nested function rebind a variable from its nearest enclosing function scope.

## 5.6 Tests and complexity

| Test | Expected result |
|---|---|
| Add tea with no quantity | One tea is added. |
| Add two sandwiches | Two sandwiches are recorded. |

| Test | Expected result |
|---|---|
| Unknown menu item | ValueError is raised. |
| Quantity zero | ValueError is raised. |
| Discount 5% | Total is reduced by five percent. |
| Call add_item twice without passing an order | Each call receives a separate new list. |

If an order contains n items, calculating the total takes **O(n)** time and **O(1)** additional space apart from the order data. Appending one item to a Python list is amortised **O(1)**.

# QUESTION 6: Recursion and Memoisation – Staircase Counting(8 Marks)

## 6.1 Required task

Count the number of ways to climb a staircase when each move can take either one step or two steps. Compare a basic recursive solution, a memoised solution and an iterative solution. Test small values and a larger value such as n = 1000.

## 6.2 Recurrence relation

Let W(n) represent the number of ways to reach step n. The final move must have come from step n–1 or n–2. Therefore, **W(n) = W(n–1) + W(n–2)**, with base cases **W(0) = 1** and **W(1) = 1**.

## 6.3 Naive recursive solution

```
def ways_recursive(n):
    if n < 0:
        return 0
    if n == 0 or n == 1:
        return 1
    return ways_recursive(n - 1) + ways_recursive(n - 2)

for n in range(1, 7):
    print(n, ways_recursive(n))
```

This version is easy to understand but repeats the same subproblems many times. Its time complexity grows exponentially, approximately O(2), and its recursion depth is O(n).

## 6.4 Memoised recursive solution

```
from functools import lru_cache

@lru_cache(maxsize=None)
def ways_memo(n):
    if n < 0:
        return 0
    if n == 0 or n == 1:
        return 1
    return ways_memo(n - 1) + ways_memo(n - 2)

for n in range(1, 7):
    print(n, ways_memo(n))

print("Ways for 1000 steps:", ways_memo(1000))
```

Memoisation stores the answer for each input value. When the same subproblem is needed again, its stored result is returned instead of recalculating it. Time complexity is O(n) additions and the cache uses O(n) entries. The recursive version still uses O(n) call-stack depth and can exceed Python's recursion limit for sufficiently large n.

## 6.5 Iterative solution

```
def ways_iterative(n):
    if n < 0:
        return 0
    if n == 0 or n == 1:
        return 1
```

```
    previous_two = 1  # W(0)
    previous_one = 1  # W(1)

    for _ in range(2, n + 1):
        current = previous_one + previous_two
        previous_two = previous_one
        previous_one = current

    return previous_one

for n in range(1, 7):
    print(n, ways_iterative(n))

print("Ways for 1000 steps:", ways_iterative(1000))
```

### 6.6 Expected values for n = 1 to 6

| Number of steps (n) | Number of ways W(n) |
|---|---|
| 1 | 1 |
| 2 | 2 |
| 3 | 3 |
| 4 | 5 |
| 5 | 8 |
| 6 | 13 |

### 6.7 Comparison

| Approach | Time complexity | Auxiliary space | Main point |
|---|---|---|---|
| Naive recursion | O(2) approximately | O(n) call stack | Simple but repeats work. |
| Memoisation | O(n) additions | O(n) cache and stack | Avoids repeated subproblems. |
| Iteration | O(n) additions | O(1) stored values | Efficient and avoids recursion depth. |

The exact bit-operation cost increases as the result becomes a larger integer. The O(n) analysis counts arithmetic operations, not the bit-level cost of adding very large integers.

# QUESTION 7: Diagnose, Redesign and Validate (13 Marks)

### 7.1 Selected task

Thestaircase-countingtask from Question 6 is selected for diagnosis and redesign. It provides a clear example of an initiallycorrectbutinefficient recursivesolution.

### 7.2 Diagnosis of the original approach

The naive recursive functionc alls itself forn1a–nd n–2. These branches overlap: for example, the value for a smaller step can be calculated repeatedly. As n increases, the number of function calls grows rapidly. A recursive implementation can also reach Python's recursion-depth limit for a large input.

### 7.3 Redesign

Theiterative method keeps only the two previous results. It computes each next value once, updates the two stored values, and avoids recursive function calls. It is therefore suitable for large n such as 1000.

```
def staircase_count(n):
    if not isinstance(n, int):
        raise TypeError("n must be an integer")
    if n < 0:
        raise ValueError("n cannot be negative")
    if n == 0 or n == 1:
        return 1

    two_steps_back = 1
    one_step_back = 1
```

```
    for _ in range(2, n + 1):
        current = two_steps_back + one_step_back
        two_steps_back = one_step_back
        one_step_back = current

    return one_step_back


# Validation
for value in range(0, 7):
    print(f"n={value}: {staircase_count(value)}")

print("n=1000 calculated successfully.")
print("Number of digits:", len(str(staircase_count(1000))))
```

## 7.4 Validation table

| Test input | Expected result / behaviour | Purpose |
|---|---|---|
| n = 0 | 1 | Checks the empty staircase base case. |
| n = 1 | 1 | Checks the one-step base case. |
| n = 2 | 2 | Checks both possible moves. |
| n = 5 | 8 | Checks a normal input. |
| n = 6 | 13 | Checks the next recurrence result. |
| n = 1000 | A large integer result is produced | Checks that iteration avoids recursion depth. |
| n = -1 | ValueError | Rejects a negative number of steps. |
| n = 2.5 | TypeError | Rejects a non-integer input. |

## 7.5 Measuring performance

```
import time

def time_function(function, n):
    start = time.perf_counter()
    result = function(n)
    elapsed = time.perf_counter() - start
    return result, elapsed

# Use a modest n for naive recursion; do not benchmark it at n=1000.
for n in (10, 20, 30):
    result, elapsed = time_function(ways_recursive, n)
    print(f"Naive recursion n={n}: {elapsed:.6f}s")

for n in (10, 20, 30, 1000):
    result, elapsed = time_function(staircase_count, n)
    print(f"Iteration n={n}: {elapsed:.6f}s")
```

The program measures the execution time on the computer where it is run. Results vary by device and Python version. The naive method is intentionally measured only for modest input sizes because its work increases exponentially.

## 7.6 Justification

Theredesigned iterative algorithm has O(n) arithmetic operations and O(1) auxiliary storage for the two previous values. It avoids the repeated work of naive recursion and the call-stack limit associated with recursive implementations. The memoised method also reduces repeated calculations to O(n), but stores a cache and still uses recursive calls.

## 7.7 Limitations

The program counts possible step sequences under the assumption that each move is exactly one or two steps and that every sequence is considered distinct by its order. It does not model blocked steps, variable step sizes or restrictions on consecutive moves. For extremely large n, the integer result itself becomes very large, so arithmetic and memory costs for the integer representation also increase.